Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

.NET Core Web API

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- ASP and ASP.NET
- RESTful implementation in Web API
- Web API basics
- Creating Controllers
- Controller Actions and Options
- Creating Models
- Routing
- Consuming Resources from a Web API

UNIVERSITY of HULL

Recap

- Recall our last session where we looked at ROA and RESTful architectures
  - REST is an architectural design for services offered over the web
  - The implementation of the architectural design is not defined
- Web API was introduced as a possible framework for producing a RESTful application
- Today we are going to look in some detail at Web API
- This should be particularly important to you as you are expected to use it for your coursework!

UNIVERSITY of HULL

ASP.NET

- In 1998 Microsoft released ASP, a server-side scripting language
- In 2002, Microsoft scrapped ASP and replaced it with a server-side web application framework called ASP.NET. In 2016 Microsoft updated this to ASP.NET Core
- It was designed to produce dynamic websites, web applications and web services.
- It is in direct competition with other Web Application Frameworks such as:
  - Laravel
  - Express.js (based on Node.js)
  - Django
  - Ruby on Rails
  - Cold Fusion
  - Etc. etc. etc.

ASP.NET Web API

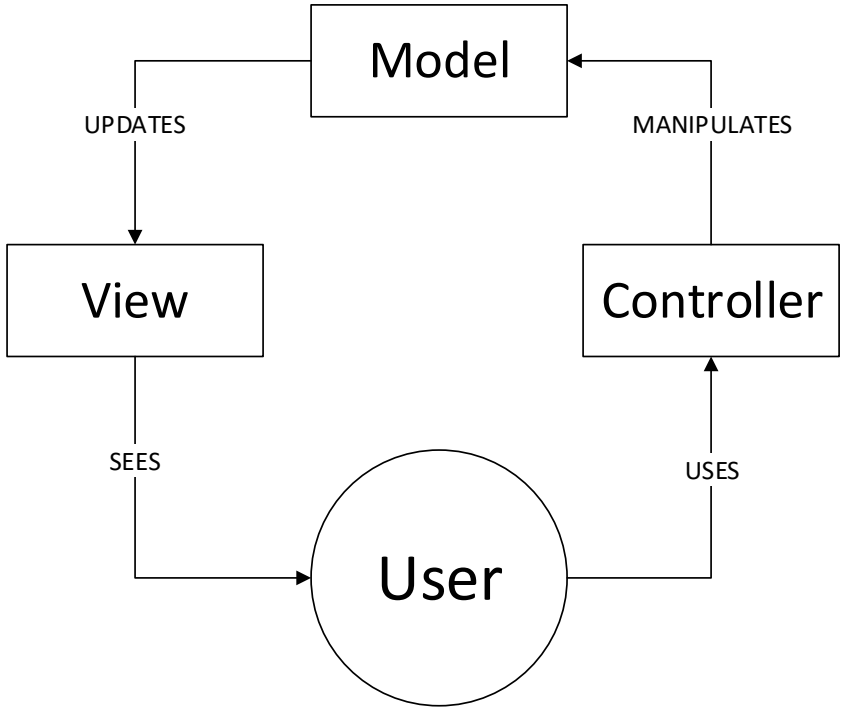
- There are lots of options for ASP.NET development
  - ASP.NET Web Pages
  - ASP.NET MVC
  - ASP.NET Web Forms
  - ASP.NET Framework Web API
  - And the one we are interested in: **ASP.NET Core WebAPI**

- We are interested in ASP.NET Web API because it offers us an easy pre-built implementation of a RESTful architecture.

UNIVERSITY of HULL

ASP.NET Web API vs MVC

- The Model View Controller pattern (MVC) is an established pattern that aids abstraction between the DATA (model), the COMMANDS (controller) and the OUTPUT (view)



- Web API can be thought of as similar to MVC but without the View
- We'll return to MVC in a future lecture

UNIVERSITY of HULL

ASP.NET Web API vs ASP.NET Minimal API

Key differences:

- **Syntax:** Minimal APIs have a simplified syntax, Web API uses a more traditional MVC-style

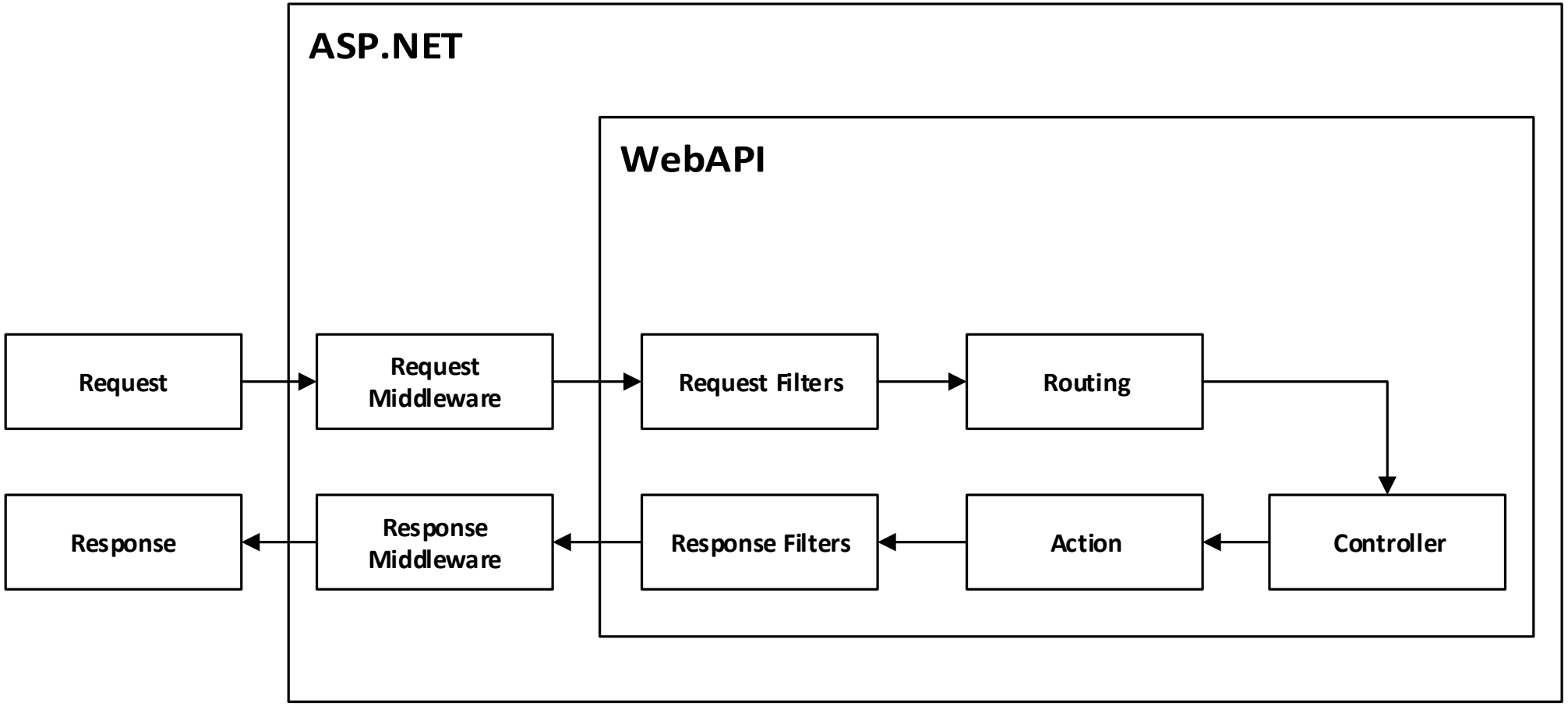
- **Routing:** Minimal APIs have a new routing syntax that allows developers to map HTTP methods and URL patterns directly to a single endpoint. Web API uses a more traditional attribute-based routing approach.

- **Configuration:** Minimal APIs provide fewer configuration options than Web API. This can make them quicker to set up and easier to use for simple APIs, but may limit their flexibility for more complex scenarios. This is the main reason we will focus on Web API.

- **Size and performance:** Minimal APIs are smaller and faster than Web API. This can make them a good choice for lightweight APIs or microservices.

Learn Minimal APIs: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis

ASP.NET Web API Pipeline



UNIVERSITY of HULL

ASP.NET Web API Controller

- Recall our CRUD operations (**C**reate, **R**etrieve, **U**pdate, **D**elete)

- We can map these operations to HTTP protocols in the following manner:

- **GET :** Retrieve - requests and retrieves the data from a remote resource
- **PUT :** Update - updates the current data on the remote server
- **POST :** Create - takes a new piece of data and creates a place for it on the server
- **DELETE :** Delete - removes data from the server

- It is the responsibility of our Controller to facilitate these operations

- ***Recall… Why are these operations so important?***

Because they map directly to DB operations!

UNIVERSITY of HULL

ASP.NET Web API Controller

A controller in Web API inherits from the **`ControllerBase`** type inside the **`Microsoft.AspNetCore.Mvc`** namespace:

```
public class DoWorkController : ControllerBase
```

This type contains methods and properties which manage HTTP requests and responses

You can create as many controllers as you like. Conventionally, a controller is named **`<title>Controller`**. When calling the controller via an HTTP request, the title only is used.

So for the above example we could call **`www.mydomain.com/DoWork`**

UNIVERSITY of HULL

ASP.NET Web API Controller

Controllers contain the actions that our API can perform.

Actions (GET, PUT, POST, DELETE) are our methods mapped to CRUD operations

We must define which HTTP Verb our Action will respond to:

HTTP Attributes allow you to define what the action is.

They are: `[HttpGet]` `[HttpDelete]` `[HttpPost]` `[HttpPut]`

You can declare lots of `GET` operations with the same parameters in the same controller

```
[HttpGet]
public string SayHello(){ return "Hello World"; }
[HttpGet]
public string SayGoodbye(){ return "Goodbye World"; }
```

UNIVERSITY of HULL

ASP.NET Web API Controller

Action Methods with Name Attributes

Sometimes, you may want to name your actions explicitly, especially when want to use the same method name twice.

For example, you may want two Actions: `SayHello` and `SayGoodbye`

```
[ActionName("SayHello")]
public string Get(){ return "Hello World"; }

[ActionName("SayGoodbye")]
public string Get([FromQuery]string username)
{ return "Goodbye" + username; }
```

UNIVERSITY of HULL

ASP.NET Web API Controller

But remember, even with using Name Attributes you still can't do this:

```
[ActionName("SayHello")]
public string Get(){ return "Hello World"; }

[ActionName("SayGoodbye")]
public string Get(){ return "Goodbye World"; }
```

…because you still can't have two identical method signatures

UNIVERSITY of HULL

ASP.NET Web API Controller

Our client would call our actions like this:

www.mydomain.com/DoWork/SayHello

www.mydomain.com/DoWork/SayGoodbye

In the case where SayGoodbye was declared with a parameter, you may have noticed that we were receiving a string parameter ‘`username`’. Let’s look at that again:

```
[ActionName("SayGoodbye")]
public string Post([FromQuery]string username)
{ return "Goodbye" + username; }
```

We would normally expect some data to be sent to the server as part of the URL from the client.

ASP.NET Web API Controller Request Parameters

When receiving parameters, we have some options and we use attributes to declare where to get the parameters from. We'll investigate 3 key attributes:

`[FromBody]` : Grabs parameters from the message Body

`[FromQuery]` : Grabs parameters from the URI

`[FromHeader]` : Grabs parameters from the Header

All actions can receive and use parameters, including GET

All actions can also work with no parameters

Although usually only GET/DELETE actions have no parameters. **Why?**

UNIVERSITY of HULL

ASP.NET Web API Controller Request Parameters

Web API has default functionality if you don't include a parameter attribute:

- If the parameter is a simple type (one which can be easily cast to a string and parsed back) it reads from Query
- If the parameter is any complex type, Web API tries to read from the message BODY
- When reading from the BODY, the header is used to determine the format of the message. Conventionally XML or JSON
  - But WebAPI will make a guess if it's not set

UNIVERSITY of HULL

ASP.NET Web API Controller URI Request Parameters

`[FromQuery]` Requires parameters to be sent in through the URI

How can we do this?

Last lecture we looked at URI composition. In this case ***we use queries***. Given the action:

```
[ActionName("SayGoodbye")]
public string Get([FromQuery]string username)
{ return "Goodbye" + username; }
```

We can use this method remotely by calling:

`www.mydomain.com/DoWork/SayGoodbye?username=PostmanPat`

And we get the result:

`Goodbye PostmanPat`

UNIVERSITY of HULL

ASP.NET Web API Controller Header Values

The header is the final place where we can get values from the client for use in the server,

These can be passed in as parameters

```
public string Get([FromHeader(Name="Accept-Language")]string language, [FromQuery]string username)
```

This is new in .NET Core

UNIVERSITY of HULL

ASP.NET Web API Controller Header Values

OR

Instead we can query the header via the request we have been passed:

```
this.Request.Headers.TryGetValue("Accept-Language", out var languages);
```

You can create as many header values as you like, although there are also standard ones which are defined for the HTTP protocol, e.g. content-type, user-agent, accept-language

- As you can submit many values for one header (delimited by a comma) languages is an IEnumerable collection

UNIVERSITY of HULL

ASP.NET Web API Controller Values

In fact, we can access Queries, the Body and Headers using this.Request:

```
this.Request.Query.TryGetValue("My-Query-Name", out var value);
this.Request.Headers.TryGetValue("My-Header-Value", out var value);
```

Body is a little different, because it is managed as a stream:

```
using (var reader = new StreamReader(this.Request.Body))
{
    var body = reader.ReadToEnd();
}
```

Why a stream? Because the size of Body is (in theory) infinite and usually can be read as it arrives rather than only reading it once it has all arrived. Useful with a large body or slow connection.

Example:

Request

www.mydomain.com/DoWork/SayGoodbye

Request Header

Username: Danger Mouse, Micky Mouse, Mighty Mouse

Code

```
[ActionName("SayGoodbye")]
public string Get()
{
    this.Request.Headers.TryGetValue("Username", out var value);
     return "Goodbye" + value.ElementAt(0);
}
```

Result

Goodbye Danger Mouse

UNIVERSITY of HULL

ASP.NET Web API Controller Response

We've seen method signatures which define a return type.

e.g. `public string Get()`

In these situations the returned value is passed in the message body.

However, we may still want to pass custom header values back to the client, and just generally have more control over the response message that is sent (e.g. the status)

This can be done by returning an object that implements `IActionResult`, e.g…

```
public IActionResult Get()
{
    return Ok("Hello World");
}
```

This returns the status OK with the string "Hello World" as the body

UNIVERSITY of HULL

ASP.NET Web API Controller Response

Sometimes we want to change the response or have more control over it.

We can do this by changing the properties of this.Response

This can be done by returning a different implementor of `IActionResult`, e.g…

```
public IActionResult Get()
{
    this.Response.StatusCode = 501;
    return new ObjectResult("Not Implemented");
}
```

This returns the status 501 with the string “Not Implemented” as the body

There are a number of other Result classes which implement from IActionResult

UNIVERSITY of HULL

ASP.NET Web API Model

- Once we've received a request, we often need to access data or perform some calculation

- As with most programming scenarios, we should maximise code reuse and improve maintainability.
  - Web API does this through the use of abstraction between the controller and model.
  - We should also practice good technique by abstracting server functionality away from the controller and to the model (and helper objects that the model uses).

- As well as logic, the Model is concerned with providing access to data.

- This usually means facilitating access to and manipulation of an underlying database

(although the data structure could be implemented in any way we choose)

UNIVERSITY of HULL

Introducing Entity Framework

- Often, in a ASP.NET Web API system, the model will use **Entity Framework**, a Microsoft **relational database** tool, designed to work with **SQL**.

- Entity Framework can:
  1. create a database for you, based on programmer-defined objects in code; or
  2. create a database for you, based on a database schema that you define; or
  3. use an existing database and (if required) create the code for you that allows database tables to be converted to code objects

– but it is important to remember that the database is another layer, separate from the Model, and is often on another machine altogether.

UNIVERSITY of HULL

Introducing Entity Framework

- However it is implemented, Entity Framework allows table rows to be accessible to the developer as IEnumerable collections
- You can then manipulate the objects created by the Entity Framework, in the same way as you might manipulate any C# object stored in a collection.
- Upon making a change, you simply call SaveChanges() on the collection to cause a database update.
  - New, modified or deleted objects are compared against the database and the database updated to reflect your changes.
- We will discuss Entity Framework more next session.

Remotely Accessing a ASP.NET Web API

- Once our Controller and Model logic is finished, our API is ready to be released upon the world, but there's one last major hurdle…
- The client must be able to access our API using a URI but the format of our resource identifiers may be abnormal.
- Web API requires us to define the structure of expected URI's so it knows how to correctly route a request to the particular controller and action that we have defined.
- This is known as **Routing** or **Route Mapping**

ASP.NET Web API Route Mapping

By default, a new Web API will contain a Controller. At the top of the controller is a route:

```
[Route("api/[Controller]")]
```

This code defines the template for routing requests to this specific controller

However, in this case, a named action (e.g. `SayGoodbye`) would not function. Routing is completely up to us, so we can allow named actions by changing our `Route` to:

```
[Route("api/[Controller]/[Action]")]
```

UNIVERSITY of HULL

Putting it all Together

- **So, as an example…**
- We have a Controller called DoWorkController

```
public class DoWorkController : ApiController
```

- We have an Action called SayHello, it takes a parameter user_ID from the URI and should return `"Hello " + username`:

```
[HttpGet]
public string SayHello([FromQuery] string user_ID)
```

- Our routing is set up to expect the format:

```
api/[controller]/[action]
```

- We have a model that provides database access, allowing us to look up the given user_ID and providing a user object (complete with username)

Query or ID?

- **Note:**

```
[HttpGet]
public string SayHello([FromQuery] string user_ID)

api/{controller}/{action}
```

  - We would use a query: *http://www.mydomain.com/controller/SayHello?user_ID=12*

```
[HttpGet]
public string SayHello(int id)

api/{controller}/{action}/{id}
```

These are linked!

  - In this case we are using an id. We would get the parameter from routing, using: *http://www.mydomain.com/controller/SayHello/12*
  - *Arguably more RESTful – also less clear in ASP.NET*

UNIVERSITY of HULL

Putting it all Together

- A client makes a GET request to…
  www.mydomain.com/DoWork/SayHello?user_ID=17389

- The server spins up a new thread to manage the request. The routing kicks in to route the request to the correct controller and action
- The controller action uses the model to search for a user with the ID: **17389** in the database
- The model returns the username for the ID: **Walter_White**
- The action forms the response with `"Hello " + username` in the message body and returns it – the Web API back end takes over and sends the response to the client.
- The server thread exits and the server forgets about the requester (stateless)

- The client receives the response: **Hello Walter_White**

UNIVERSITY of HULL

Resources

- There will be a lab on Web API.

- Dedicated ASP web resource:
  - https://www.asp.net/web-api

https://docs.microsoft.com/en-us/aspnet/core/tutorials/first-web-api

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- ASP and ASP.NET
- RESTful implementation in Web API
- Web API basics
- Creating Controllers
- Controller Actions and Options
- Creating Models
- Routing
- Consuming Resources from a Web API

In the next session we will look at Databases and Entity Framework

UNIVERSITY of HULL